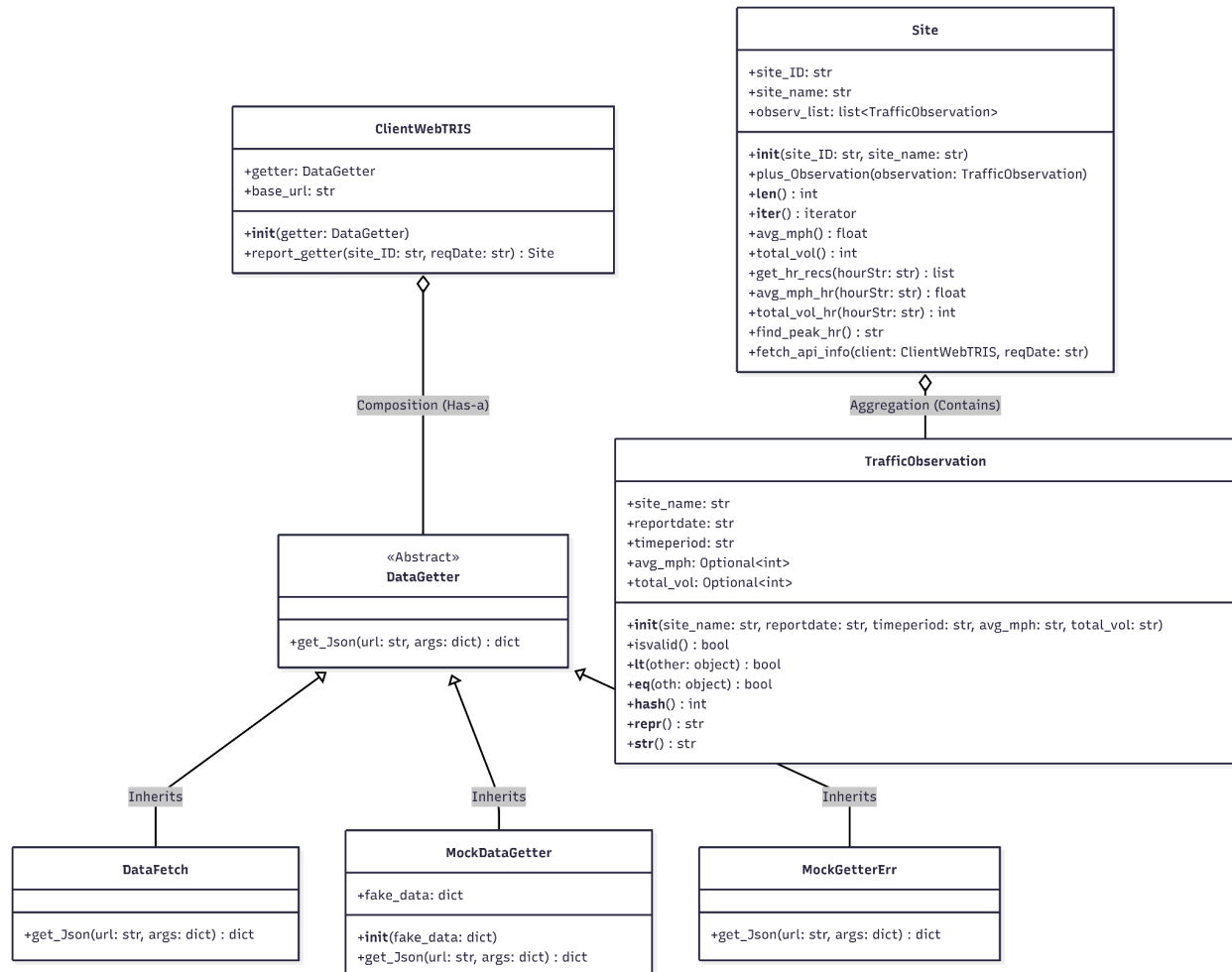


WebTRIS Client Application Design Report
Dr. Amil Mohanan
25/3/2026



In my process of designing the WebTRIS client application, my goal was to create an adaptable object oriented program that could maneuver through the real-world unpredictable API data while also remaining extendable for our future AE3. In the UML diagram above, my system architecture is purposely isolated into 3 main areas to address different issues.

At the bottom level, my `TrafficObservation` class handles the data representation which aggregates them in the `Site` class. This is displayed through diamond aggregation between `Site` and `TrafficObservation` on the UML diagram. For the networking infrastructure, I defined the abstract `DataGetter` class using a Strategy Pattern. My `ClientWebTRIS` class used composition to intake any of the getter strategies during the

runtime. As a result, the client never directly engages with the network, but rather assigns to whatever confirmed DataGetter implementation was initialized with. As such, these clear differences encouraged me to design the rest of my application following this format.

Coming from a programming background in Java, I'm generally inclined to creating explicit getter methods and my own sorting methods [ie. `site.get_size()` or `observation.compare_to()`] for the modeling for the data. Despite this, by using some of the concepts of sequential protocol and the operator overloading from the week 4 lecture material, I was able to incorporate these behaviors directly into my objects.

I directly tried to architect my TrafficObservation and Site classes to interact with Python's native environment. Through my implementation of the `__iter__` and `__len__` methods in the Site class, you could use a traditional for loop or `len(site)` to abstract away from internal list management. Overriding `__lt__` in the TrafficObservation class lets the built-in `sorted()` method sort the records in chronological order natively. My implementations of `__hash__` and `__eq__` guarantee that those objects kept a distinctive identity and could be properly inserted into dictionaries or sets. Looking at AE3 as well, some of the objects could be imported and would be assigned weights in a graphical representation of the M25 roadway system. Since the classes here perform as Python native sequences, the workflow for figuring out the edge weight for BFS or Dijkstra's algorithm will be more efficient without introducing too many additional iteration features.

When analyzing the live WebTRIS API outputs, they do appear incoherent and it remained common for the API to return ("") or empty strings for the Total Vol and Avg mph falling under an edge case scenario. For cases such as these, I generally aim to build high-reliability environments for data, typically for tracking infrastructure. While my constructor could try to cast those empty strings using `int("")`, my application programs would likely crash encountering a `ValueError`. In order to handle this, I created a conditional logic flow to find empty string occurrences and store them as `None`. This explicitly signals the likelihood for using the `Optional[int]` type hint from the typing module. I then encapsulated my logic for the validation in `isvalid()` function to keep the `None` data stored from hampering my math logic. My Site class also looks to remove any of the uncompleted records prior to running any accumulator loops. Hence, this means the end-user running `main()` function can average together the valid records instead of crashing because some intervals of recorded information were missing or invalid data.

Since the requirements cited a feature to swap fake data and live API data for the test cases, the direct approach would have been to brute-force an `[if testing_mode == True:]`

statement within the ClientWebTRIS class. However, in the week 6 lecture material, this goes against fundamentals of OOD and every occurrence a new data source is added, the main client logic and math would need to be altered again. My solution was to implement a Strategy Pattern using the abstract base class – DataGetter. By using composition, the constructor can obtain the fetching strategy for the client. Due to having a has-a correlation, the ClientWebTRIS class doesn't have to know if it is fetching data from the live API or the local dictionaries.

By separating these issues, I could use the MockDataGetter to separate the math logic and JSON parsing from outside network variations and changes. By looking ahead at AE3, I noticed that the brief mentioned a local file containing sample data would be given to implement the graph network isolated from the live WebTRIS API. This encouraged me to implement my strategy pattern, which can incorporate local files much more efficiently. As a result, I would just need to implement an inheritance instance of a FileGetter class that inherits DataGetter logic, thus keeping the main client from being changed excessively.

Architecting my application infrastructure so that it functions under no errors, missing data, or network failures was straightforward. Since network requests remain relatively unstable, if my application doesn't have countermeasures in place ahead of time, it renders the end-user searching for Python tracebacks. My program design isolates the error detection aside from the addressing of an error. Low-level exceptions such as HTTPError or Timeout are raised by the DataFetch class and are addressed with descriptive messaging. This gives the ability for the exception to propagate through the hierarchy to the main.py file. I purposely positioned the last try/except code block within the main.py file instead of getting rid of the error in the client. This allows for the integrated program the ability to choose which to proceed with. In order to confirm this, I built the MockGetterErr which solely raises a Timeout exception. This lets me implement pytest.raises to show in the test case that my client transmits errors without being required to physically disconnect from the internet.

During the building phase, I envisioned 2 different end users that would employ my application. A standard user who runs the main() method to obtain traffic data could use the data cleaning and exception bubbling features I implemented to ensure an efficient workflow. During network failures or drops, they would be returned a warning opposed to having to search through Python tracebacks. Even in the event a sensor drops offline, they could still receive correct calculated averages from the still-valid data that remained.

The second envisioned end-user would be another developer, either a classmate or myself who is working on the AE3 for this project. Since we are likely importing the

WebTRIS application codebase for AE3, the objects will be used to give weighted edges in the M25 Roadway system's graphical representation. Since I architected the classes as a native python sequence, the overall iterative workflow through the record data to find weights for BFS or Dijkstra's algorithm will fall into place more optimally. Going further, a future developer/user can just inherit a new FileGetter from the DataGetter class without altering the logic of the client excessively.